

What PHP Developers Can Do to Secure Web Apps

Web applications scripted in PHP are vulnerable if proper precautions are not taken to ensure security. This article highlights the measures to be taken to reduce and eliminate these vulnerabilities, which generally arise due to negligent programming. An ounce of prevention is better than a pound of cure, the proverb says. For developers, no amount of caution is too much!

I've seen many people abandon PHP for security reasons. Many do this just because someone else told them it is not secure, but some have experienced the hard reality of attackers taking control of their websites and doing severe damage (even financially).

So, what's the truth? Is PHP inherently vulnerable? Well, it is true that PHP has many pitfalls, but nobody will deny the fact that it is one of the most friendly Web development tools out there, and even some of the busiest websites on the planet (including Wikipedia) run on it. These have no particular issues that make them more vulnerable compared to sites developed in other languages.

As with most 'bad things', PHP's bad reputation comes from how it is used. It is easy to write vulnerable apps in PHP, but it is easier to get cheated by the false sense of security induced by some languages and frameworks. They cover us as long as our app is a stock item, but the moment we do something out-of-the-box, we are on our own. The Web is one of the most dangerous environments for an application to run, where arrows can come from unexpected directions. You might have built the backend upon a super secure framework, but your false expectations of how the client-side works might cost you everything.

So instead of hopping from framework to framework trying to



find the most secure one, let's first try to understand and undo our mistakes. This article discusses some security considerations that are useful while building any Web app, but from a PHP viewpoint.

Injection attacks

Whenever a website processes and outputs some value from the client side, either as fresh entries or modifications of the defaults, there is a chance of attacks. The visitor can knowingly or unknowingly enter dangerous values, or a third party can conceive a malicious URL and make a legit client visit it. Such attacks fall into the injection and cross-site scripting (XSS) categories.

First, think about where the data comes from. There will be user-settable and user-editable values in `$_GET`, `$_POST` and `$_FILES`. Values in `$_COOKIE` are set by you, but can be edited by a malicious visitor. That means additional checks might be needed. Also, never set secrets and privileges in cookies. Store them in

`$_SESSION` and the database (server side). Cookies are just to identify the user and to keep extremely temporary stuff.

Cross-site scripting

Cross-site scripting (XSS) attacks include various tricks by which attackers inject code into websites without accessing or modifying the server code.

This means your users can be harmed while your server remains intact.

A simple example is your site accepting a piece of text from the user and displaying it back. If the visitor enters HTML instead of text, that gets rendered. If the visitor enters JavaScript, that gets executed. Isn't this self-induced damage? No, it can affect other users (more on this in the 'Session hijacking' section).

The solution is to escape all specials in the user-input values before displaying (e.g., `convert < to <`). There are functions like `htmlspecialchars()` in PHP for this.

File uploading

In file uploading forms, you'll have to add client-side size and type checks (usually using JavaScript) so that the client can see if the file is incompatible even before the upload starts. But be sure to add the same checks in the server-side PHP so that an attacker-modified client-side won't fool the server. But there can also be pitfalls. While you can use

